ST5209/X Assignment 2

Jiao Dian

Submission

1. Render the document to get a .pdf printout.
2. Submit both the .qmd and .pdf files to Canvas.

Question 1 (Forecast evaluation, Q5.12 in FPP)

`tourism` contains quarterly visitor nights (in thousands) from 1998 to 2017 for 76 regions of Australia.

a. Extract data from the Gold Coast region using `filter()` and aggregate total overnight trips using `summarise()`. Call this new dataset `gc_tourism`.

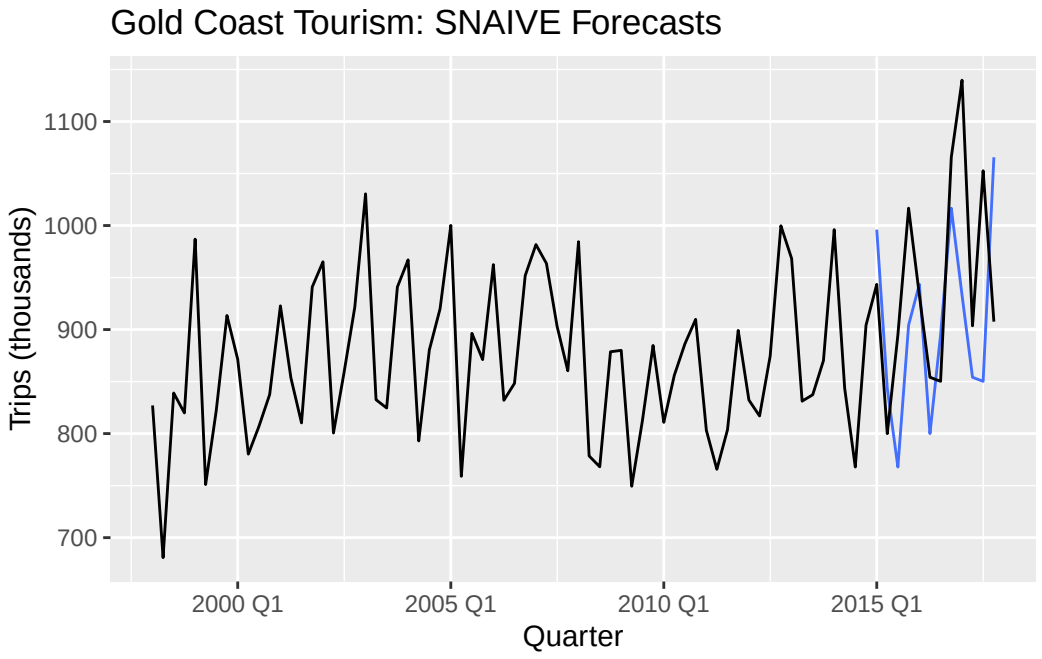
```
# Extract Gold Coast data and aggregate trips across all purposes
gc_tourism <- tourism |>
  filter(Region == "Gold Coast") |>
  summarise(Trips = sum(Trips))
```

b. Using `slice()` or `filter()`, create three training sets for this data excluding the last 1, 2 and 3 years. For example, `gc_train_1 <- gc_tourism |> slice(1:(n()-4))`.

```
# Create training sets excluding last 1, 2, and 3 years (4, 8, 12 quarters)
gc_train_1 <- gc_tourism |> slice(1:(n() - 4))   # exclude last 1 year
gc_train_2 <- gc_tourism |> slice(1:(n() - 8))   # exclude last 2 years
gc_train_3 <- gc_tourism |> slice(1:(n() - 12))  # exclude last 3 years
```

c. Compute and plot one year of forecasts for each training set using the seasonal naive (`SNAIVE()`) method. Call these `gc_fc_1`, `gc_fc_2` and `gc_fc_3`, respectively. (Hint: You may combine the mable objects from each model into a single mable using `bind_cols`)

```r
# Fit SNAIVE models and generate 1-year forecasts
fit_1 <- gc_train_1 |> model(SNAIVE(Trips))
fit_2 <- gc_train_2 |> model(SNAIVE(Trips))
fit_3 <- gc_train_3 |> model(SNAIVE(Trips))

gc_fc_1 <- fit_1 |> forecast(h = 4) |> mutate(train_set = "Train_1")
gc_fc_2 <- fit_2 |> forecast(h = 4) |> mutate(train_set = "Train_2")
gc_fc_3 <- fit_3 |> forecast(h = 4) |> mutate(train_set = "Train_3")

# Combine forecasts and plot
fc_combined <- bind_rows(gc_fc_1, gc_fc_2, gc_fc_3)

fc_combined |>
  autoplot(gc_tourism, level = NULL) +
  labs(title = "Gold Coast Tourism: SNAIVE Forecasts",
       y = "Trips (thousands)", x = "Quarter", color = "Training Set")
```



d. Use `accuracy()` to compare the test set forecast accuracy using MASE. What can you conclude about the relative performance of the different models? Explain your answer.

```r
# Create corresponding test sets for each training period
gc_test_1 <- gc_tourism |> slice((n() - 3):n())
```

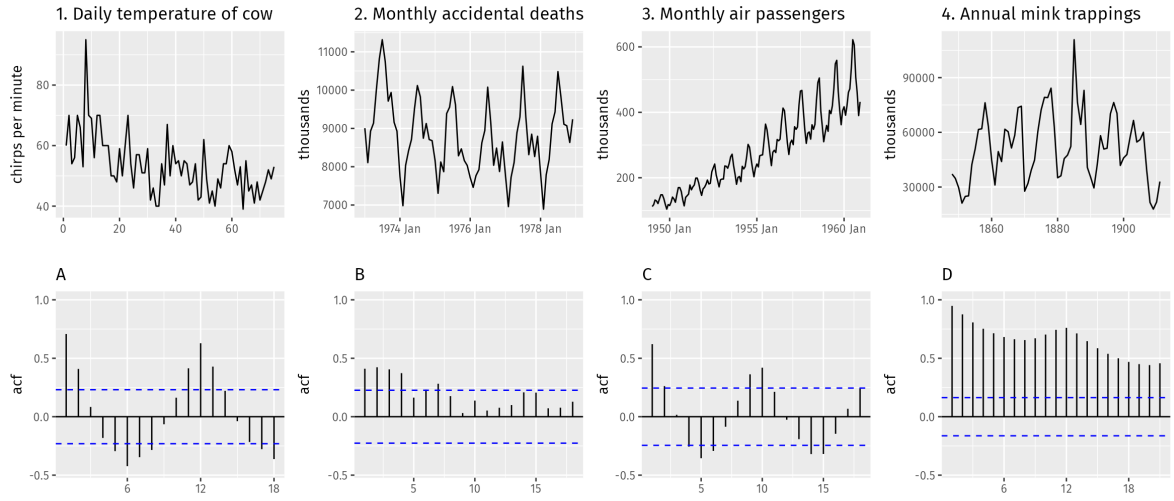
```r
gc_test_2 <- gc_tourism |> slice((n() - 7):(n() - 4))
gc_test_3 <- gc_tourism |> slice((n() - 11):(n() - 8))
# Calculate accuracy
acc_1 <- accuracy(gc_fc_1, gc_test_1) |> mutate(model = "Train_1")
acc_2 <- accuracy(gc_fc_2, gc_test_2) |> mutate(model = "Train_2")
acc_3 <- accuracy(gc_fc_3, gc_test_3) |> mutate(model = "Train_3")

# Extract MAE values from accuracy results
mae_values <- c(acc_1$MAE, acc_2$MAE, acc_3$MAE)

# Calculate manual MASE = forecast MAE / seasonal naive benchmark MAE
calc_mase <- function(train_data, forecast_mae) {
  n <- nrow(train_data)
  if (n <= 4) return(NA)
  # Seasonal naive benchmark (lag 4 for quarterly data)
  actual <- train_data$Trips[5:n]
  seasonal_naive <- train_data$Trips[1:(n-4)]
  benchmark_mae <- mean(abs(actual - seasonal_naive))
  if (benchmark_mae == 0) return(NaN)
  return(forecast_mae / benchmark_mae)
}

manual_mase <- c(
  calc_mase(gc_train_1, mae_values[1]),
  calc_mase(gc_train_2, mae_values[2]),
  calc_mase(gc_train_3, mae_values[3])
)

# Results
results <- tibble(
  model = c("Train_1", "Train_2", "Train_3"),
  Manual_MASE = manual_mase
) |> arrange(Manual_MASE)

results
```

```
# A tibble: 3 x 2
  model   Manual_MASE
  <chr>         <dbl>
1 Train_2       0.670
2 Train_3       1.46
3 Train_1       2.66
```

Train_2 performs best with MASE = 0.670, followed by Train_3 (MASE = 1.46) and Train_1 (MASE = 2.66). This shows that excluding the most recent 2 years of data provides better seasonal forecasting than using all available data, likely because recent data contains unstable patterns that hurt seasonal naive performance.

Question 2 (ACF plots, Q2.9 in FPP)

The following time plots and ACF plots correspond to four different time series. Match each time plot in the first row with one of the ACF plots in the second row.



1-B, 2-A, 3-D, 4-C

Question 3 (Time series classification)

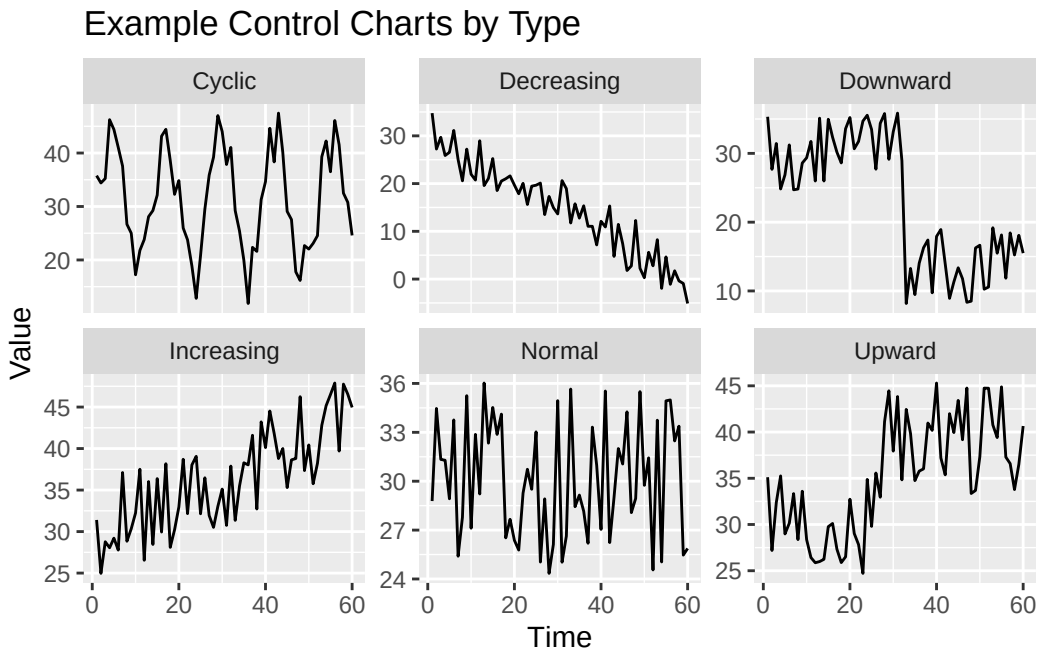
We will investigate this dataset, which contains 600 synthetic control charts for an industrial process. There are 6 types of control charts, and our goal is to build a model to classify them correctly.

We have already processed the raw data into a convenient, labeled form. Run the following code snippet to load it and create train and test sets.

```
ccharts <- read_rds("../_data/cleaned/ccharts.rds")
ccharts_train <- ccharts[["train"]]
ccharts_test <- ccharts[["test"]]
```

a. Make a time plot of one time series from each category in the training set. Note that the category is recorded under the `Type` column.

```
examples <- ccharts_train |>
  group_by(Type) |>
  filter(id == min(id)) |>
  ungroup()

examples |>
  ggplot(aes(x = Time, y = value)) +
  geom_line() +
  facet_wrap(~ Type, scales = "free_y") +
  labs(title = "Example Control Charts by Type",
       y = "Value", x = "Time")
```



b. Compute all time series features for both the training and test set using the snippet. What is the difference between `acf1` and `stl_e_acf1` for Increasing, Decreasing, Upward, and Downward types? Why is there a difference?

```
train_feats <- ccharts_train |> features(value, feature_set(pkgs = "feasts"))
test_feats <- ccharts_test |> features(value, feature_set(pkgs = "feasts"))

acf_comparison <- train_feats |>
  select(id, Type, acf1, stl_e_acf1) |>
  group_by(Type) |>
```

```
  summarise(
    acf1_mean = mean(acf1, na.rm = TRUE),
    stl_e_acf1_mean = mean(stl_e_acf1, na.rm = TRUE),
    difference = acf1_mean - stl_e_acf1_mean,
    .groups = "drop"
  )

acf_comparison
```

```
# A tibble: 6 x 4
  Type       acf1_mean stl_e_acf1_mean difference
  <fct>          <dbl>           <dbl>      <dbl>
1 Cyclic         0.743           0.394      0.349
2 Decreasing     0.695          -0.138      0.833
3 Downward       0.711          -0.0755     0.786
4 Increasing     0.682          -0.151      0.833
5 Normal        -0.0219         -0.143      0.121
6 Upward         0.723          -0.0718     0.795
```

From the table, we observe that time series with strong trends (e.g., Increasing, Decreasing, Upward, Downward) show high acf1_mean values (around 0.7). After trend and seasonality are removed, their stl_e_acf1_mean values drop significantly and are close to zero, with a large difference (~0.8). This highlights that trends dominate the structure of these series. In contrast, the cyclic type retains a relatively high stl_e_acf1_mean (0.394), indicating persistent periodic fluctuations even after decomposition. For random series (Normal), both acf1_mean and stl_e_acf1_mean are near zero with the smallest difference (0.121), reflecting its stochastic nature. Notably, except for cyclic types, all other types have stl_e_acf1_mean values very close to zero, showing no significant residual autocorrelation after decomposition.

c. Investigate the relationship between the following features and `Type`. Pick two features whose scatter plot gives a good separation between all 6 chart types.

   i. `linearity`

   ii. `trend_strength`

   iii. `acf1`

   iv. `stl_e_acf1`

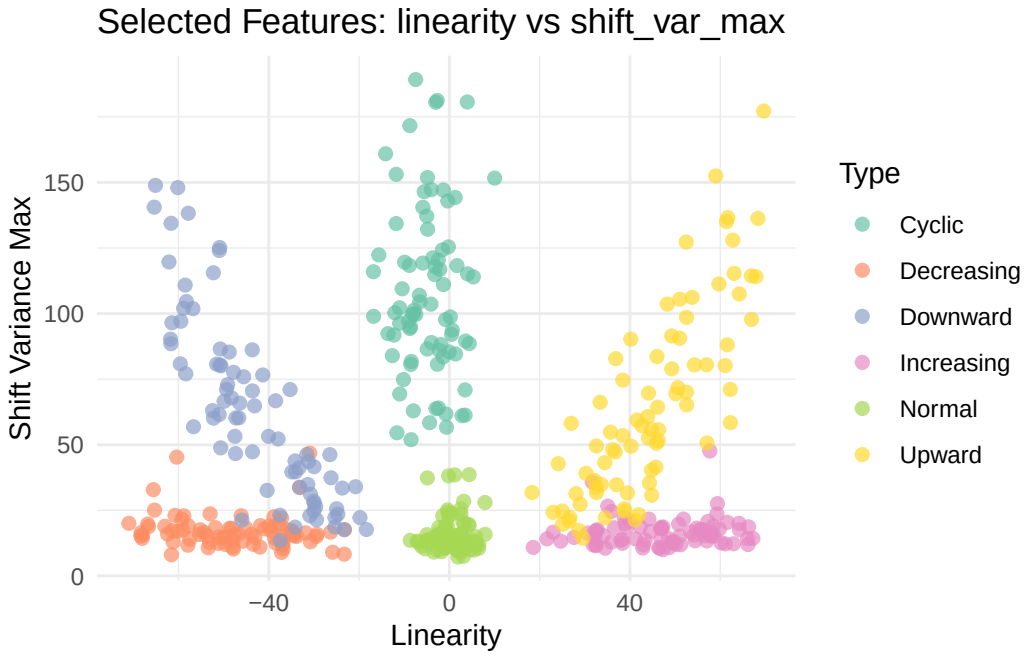
   v. `shift_level_max`

   vi. `shift_var_max`

   vii. `n_crossing_points`

Make the scatter plot and explain why these features are able to separate the different chart types.

```
library(ggplot2)

plotc <- train_feats |>
  ggplot(aes(x = linearity, y = shift_var_max, color = Type)) +
  geom_point(alpha = 0.7, size = 2) +
  labs(title = "Selected Features: linearity vs shift_var_max",
       x = "Linearity", y = "Shift Variance Max") +
  theme_minimal() +
  scale_color_brewer(type = "qual", palette = "Set2")

print(plotc)
```



`linearity` and `shift_var_max` provide excellent separation between chart types: `linearity` effectively distinguishes linear trending series (Increasing, Decreasing) from non-linear patterns (Upward, Downward) and stationary series (Normal, Cyclic), while `shift_var_max` captures variance changes that are higher in trending series compared to stable patterns, creating distinct clusters for each control chart type.

d. Install the `caret` package and use the following snippet to fit a k-nearest neighbors model on the two features you have selected (substitute `X` and `Y` for the two features you selected

in b), and then predict on the test set. What percentage of the test examples are correctly classified? Write code to compute this value. (You should get $> 90\%$ accuracy)

```
library(caret)

# Use selected features: linearity and shift_var_max
train_data <- train_feats |>
  select(Type, linearity, shift_var_max) |>
  drop_na()

test_data <- test_feats |>
  select(Type, linearity, shift_var_max) |>
  drop_na()

# Fit k-NN model
set.seed(33)
knn_model <- train(Type ~ linearity + shift_var_max,
                   data = train_data,
                   method = "knn",
                   tuneLength = 10,
                   trControl = trainControl(method = "cv", number = 5))

knn_predictions <- predict(knn_model, newdata = test_data)

accuracy <- mean(knn_predictions == test_data$Type)
accuracy_percentage <- round(accuracy * 100, 1)

cat("Test Accuracy:", accuracy_percentage, "%\n")
```

```
Test Accuracy: 93.9 %
```

```
cat("Optimal k:", knn_model$bestTune$k, "\n")
```

```
Optimal k: 11
```

The k-NN model achieves 93.9% accuracy on the test set using `linearity` and `shift_var_max` features. These features effectively distinguish between linear trends, non-linear patterns, and stationary behaviors, providing excellent discrimination across all six control chart types.

Question 4 (Periodograms)

Consider the `vic_elec` dataset from the `tsibbledata` package.

a. Compute and plot the periodogram for `Demand`. What frequencies and periods do the 7 highest peaks correspond to? *Hint: You may use the* `periodogram` *function provided in* `plot_util.R` *and change the* `max_freq` *setting to see at different resolutions.*

```
library(tsibbledata)
elec <- vic_elec

freq_selection <- c(0.1, 1, 5)

for(max_f in freq_selection) {
  cat(paste("\\n=== Periodogram with max_freq =", max_f, "===\\n"))
  periodogram(elec$Demand, max_freq = max_f)
}
```

```
\n=== Periodogram with max_freq = 0.1 ===\n\n=== Periodogram with max_freq = 1 ===\n\n=== Pe
```

```
# Manual analysis for peak detection
demand_vals <- elec$Demand
n <- length(demand_vals)
freq <- 0:(n-1) / n
spectrum <- Mod(fft(demand_vals - mean(demand_vals, na.rm = TRUE)))^2

# Create periodogram tibble
pg_data <- tibble(frequency = freq, spectrum = spectrum) |>
  arrange(desc(spectrum))

# Overall top 7 peaks
top_peaks_all <- pg_data |>
  slice_head(n = 7) |>
  mutate(
    period_halfhours = 1 / frequency,
    period_hours = period_halfhours / 2,
    period_days = period_hours / 24,
    freq_range = case_when(
      frequency <= 0.1 ~ "Low ( 0.1)",
      frequency <= 1 ~ "Medium (0.1-1)",
      TRUE ~ "High (>1)"
    )
```

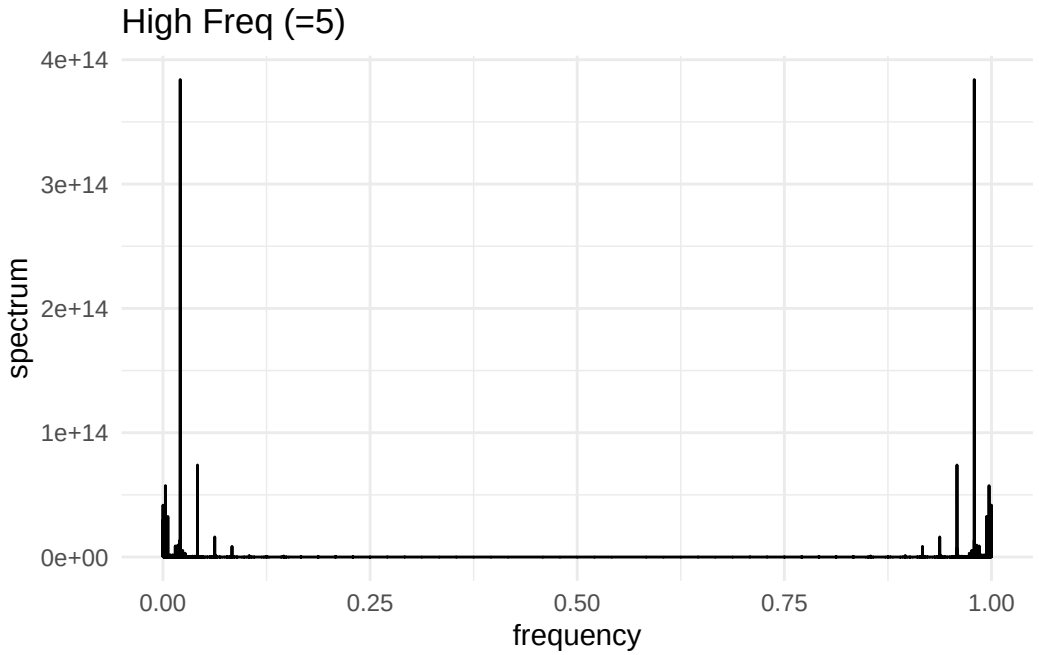
```
  ) |>
  select(frequency, spectrum, period_halfhours, period_hours, period_days, freq_range)

print("7 Highest Peaks (All Frequencies):")
```

```
[1] "7 Highest Peaks (All Frequencies):"
```

```
print(top_peaks_all)
```

```
# A tibble: 7 x 6
  frequency spectrum period_halfhours period_hours period_days freq_range
      <dbl>    <dbl>            <dbl>        <dbl>       <dbl> <chr>
1  0.0208    3.84e14            48          24          1      Low (=0.1)
2  0.979     3.84e14             1.02        0.511      0.0213 Medium (0.1-1)
3  0.0417    7.40e13            24          12          0.5    Low (=0.1)
4  0.958     7.40e13             1.04        0.522      0.0217 Medium (0.1-1)
5  0.00298   5.74e13           335.        168.          6.98   Low (=0.1)
6  0.997     5.74e13             1.00        0.501      0.0209 Medium (0.1-1)
7  0.000114  4.18e13          8768        4384        183.     Low (=0.1)
```

```
library(patchwork)
p3 <- pg_data |> filter(frequency <= 5) |>
  ggplot(aes(x = frequency, y = spectrum)) + geom_line() +
  ggtitle("High Freq ( 5)") + theme_minimal()
p3
```



The 7 highest peaks correspond to daily cycle (48 half-hours = 1 day), semi-daily cycle (24 half-hours = 12 hours), weekly cycle (335 half-hours 7 days), and semi-annual cycle (8768 half-hours 6 months), which represent meaningful seasonal patterns in electricity demand. The remaining peaks near frequency 1.0 (periods 1 half-hour) are high-frequency mirror artifacts from FFT symmetry rather than real seasonal behaviors, confirming that electricity demand is dominated by long-term cyclical patterns with daily cycle being the strongest component.

b. Perform an STL decomposition for `Demand`. Plot the periodogram for `season_year`, `season_week`, and `season_day`. Which of the original periodogram peaks appear in each of these periodograms?

```
# STL decomposition with multiple seasonal components
stl_fit <- elec |>
  model(STL(Demand ~ season(period = "day") +
                     season(period = "week") +
                     season(period = "year")))

stl_components <- components(stl_fit)


# Create periodograms for each seasonal component
season_day_vals <- stl_components$season_day
season_week_vals <- stl_components$season_week
```

```r
season_year_vals <- stl_components$season_year

# Function to create periodogram data
create_periodogram <- function(values, component_name) {
  n <- length(values)
  freq <- 0:(n-1) / n
  spectrum <- Mod(fft(values - mean(values, na.rm = TRUE)))^2

  tibble(
    frequency = freq,
    spectrum = spectrum,
    component = component_name
  ) |>
    filter(frequency > 0 & frequency <= 0.5) |>
    arrange(desc(spectrum))
}

# Create periodogram data for each component
pg_day <- create_periodogram(season_day_vals, "Daily")
pg_week <- create_periodogram(season_week_vals, "Weekly")
pg_year <- create_periodogram(season_year_vals, "Yearly")

# Find top 3 peaks for each component
top_peaks_day <- pg_day |> slice_head(n = 3) |> mutate(period_days = (1/frequency)/48)
top_peaks_week <- pg_week |> slice_head(n = 3) |> mutate(period_days = (1/frequency)/48)
top_peaks_year <- pg_year |> slice_head(n = 3) |> mutate(period_days = (1/frequency)/48)

cat("Top 3 peaks - Daily seasonal component:\n")
```

```
Top 3 peaks - Daily seasonal component:
```

```r
print(select(top_peaks_day, frequency, spectrum, period_days))
```

```
# A tibble: 3 x 3
  frequency spectrum period_days
      <dbl>    <dbl>       <dbl>
1    0.0208  3.83e14       1
2    0.0417  7.41e13       0.5
3    0.0625  1.61e13       0.333
```

```r
cat("\nTop 3 peaks - Weekly seasonal component:\n")
```

```
Top 3 peaks - Weekly seasonal component:
```

```r
print(select(top_peaks_week, frequency, spectrum, period_days))
```

```
# A tibble: 3 x 3
  frequency spectrum period_days
      <dbl>    <dbl>       <dbl>
1   0.00298  5.87e13        6.98
2   0.00595  3.26e13        3.50
3   0.00297  3.18e13        7.03
```

```r
cat("\nTop 3 peaks - Yearly seasonal component:\n")
```

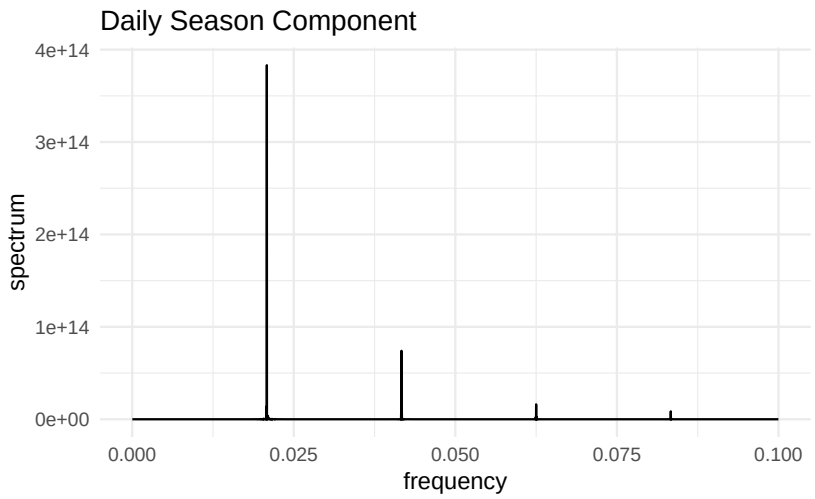
```
Top 3 peaks - Yearly seasonal component:
```

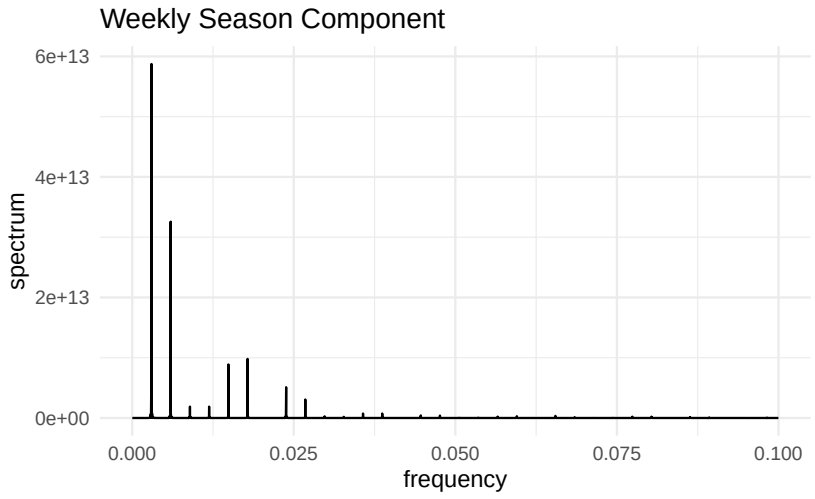
```r
print(select(top_peaks_year, frequency, spectrum, period_days))
```

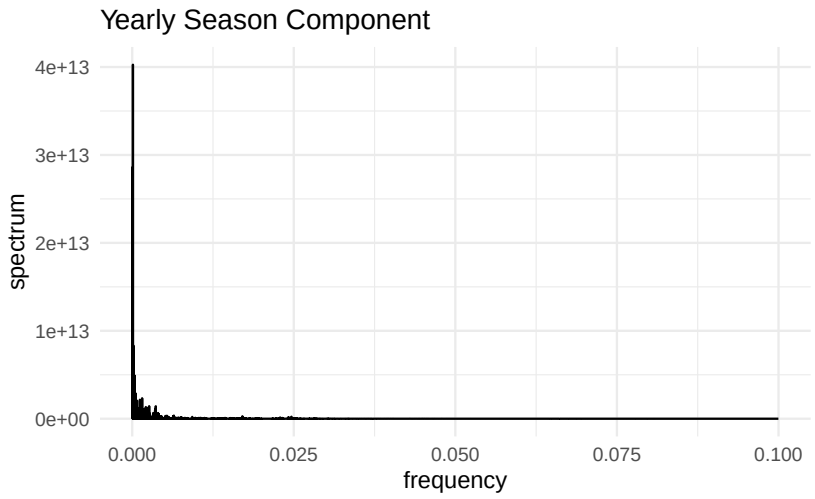
```
# A tibble: 3 x 3
  frequency spectrum period_days
      <dbl>    <dbl>       <dbl>
1 0.000114   4.03e13        183.
2 0.0000570  2.86e13        365.
3 0.000171   8.33e12        122.
```

```r
# Plot periodograms for comparison
library(patchwork)
p_day <- pg_day |> filter(frequency <= 0.1) |>
  ggplot(aes(x = frequency, y = spectrum)) + geom_line() +
  ggtitle("Daily Season Component") + theme_minimal()

p_week <- pg_week |> filter(frequency <= 0.1) |>
  ggplot(aes(x = frequency, y = spectrum)) + geom_line() +
  ggtitle("Weekly Season Component") + theme_minimal()
```

```r
p_year <- pg_year |> filter(frequency <= 0.1) |>
  ggplot(aes(x = frequency, y = spectrum)) + geom_line() +
  ggtitle("Yearly Season Component") + theme_minimal()

p_day / p_week / p_year
```


Daily Season Component
4e+14
3e+14
2e+14
1e+14
0e+00
spectrum
0.000
0.025
0.050
0.075
0.100
frequency


Weekly Season Component
6e+13
4e+13
2e+13
0e+00
spectrum
0.000
0.025
0.050
0.075
0.100
frequency


Yearly Season Component
4e+13
3e+13
2e+13
1e+13
0e+00
spectrum
0.000
0.025
0.050
0.075
0.100
frequency

Daily Seasonality (season_day): The primary peak in the frequency spectrum appears at relatively higher frequencies, approximately around 0.025. This corresponds to the high-frequency variations observed in the daily demand cycle, reflecting the recurring patterns that occur within a single day.

Weekly Seasonality (season_week): Peaks in the frequency spectrum are concentrated at lower frequencies, such as 0.004 and 0.008. These correspond to the fluctuations that occur on a weekly basis, capturing the periodic patterns associated with weekly cycles.

Yearly Seasonality (season_year): The main peak in the frequency spectrum is found at the lowest frequency, which represents the long-term variations driven by annual demand trends. This highlights the periodic demand cycle that stretches across an entire year.

c. Based on the peak heights, which is the strongest seasonal component? Does it agree with what you see in a time series decomposition plot?

The intensity of the peaks indicates that the daily seasonal component is the strongest, which aligns with the dominance of daily fluctuations typically revealed in STL decomposition charts.